

Customer Support Ticket Analytics Platform

1) Project overview

The Customer Support Ticket Analytics Platform is an end-to-end, cloud-based data engineering pipeline built for the AWS track of the Data Engineering Fellowship. The project addresses a real operational gap: an international software company's helpdesk team has years of ticket history sitting in raw CSV exports, with no consolidated way to see ticket volume, resolution performance, or agent workload. Support leadership has been relying on manual spreadsheet work, which doesn't scale and offers no trend visibility.

The pipeline ingests two real, linked source files — `issues.csv` and `issues_change_history.csv`, sourced from the Mendeley "Help Desk Tickets" dataset (Jan 2016–Mar 2023, ~60,000+ tickets) — and processes them through a fully automated AWS-native architecture: Amazon S3 (raw and curated zones), AWS Lambda (event-driven ETL trigger), PySpark (transformation and metric engineering), Amazon RDS PostgreSQL (analytics-ready storage), and Matplotlib/Seaborn/Streamlit (visualization layer).

Architecturally, the pipeline reconstructs each ticket's current state — assignee and status — from the change-history log rather than assuming it from the static issues file, which mirrors how real production support systems behave. From there, PySpark computes resolution-time metrics (`resolution_duration_hours`, `resolution_target_met`) and loads a star schema (`fact_tickets` plus `dim_agents`, `dim_customers`, `dim_projects`) into PostgreSQL, alongside three pre-aggregated analytics tables.

The platform was designed around three core business use cases: resolution performance analysis (SLA-style compliance by priority), agent workload and performance (ticket distribution and efficiency by agent), and ticket category trends (recurring or fast-growing issue categories). Each use case follows a consistent pattern — ETL logic, an output table in RDS, and a dashboard view — so the technical pipeline maps directly back to a business question support leadership actually needs answered.

2) Architecture design :

```
issues.csv / issues_change_history.csv
|
v
Amazon S3 (raw zone)
|
v
PySpark ETL
|
v
Amazon S3 (curated zone)
|
v
AWS RDS (PostgreSQL)
|
v
Matplotlib / Seaborn / Streamlit
|
v
Support Operations Dashboard
```

3) Dataset details

3.1 Source & volume

Feature	Description
Dataset name	AWS Support Ticket Data
Source	Internal Support System
Description	Historical AWS support tickets tracking lifecycle from creation to resolution
Record count	~60,000+ tickets
Raw files	<code>issues.csv</code> , <code>issues_change_history.csv</code>

3.2 What the data represents

Tickets include information such as:

- Ticket creation and resolution timestamps
- Priority / severity

- Issue type / category
- Assigned agent (or owner)
- Change history events (status changes, reassignments, etc.)

Primary analytics goals:

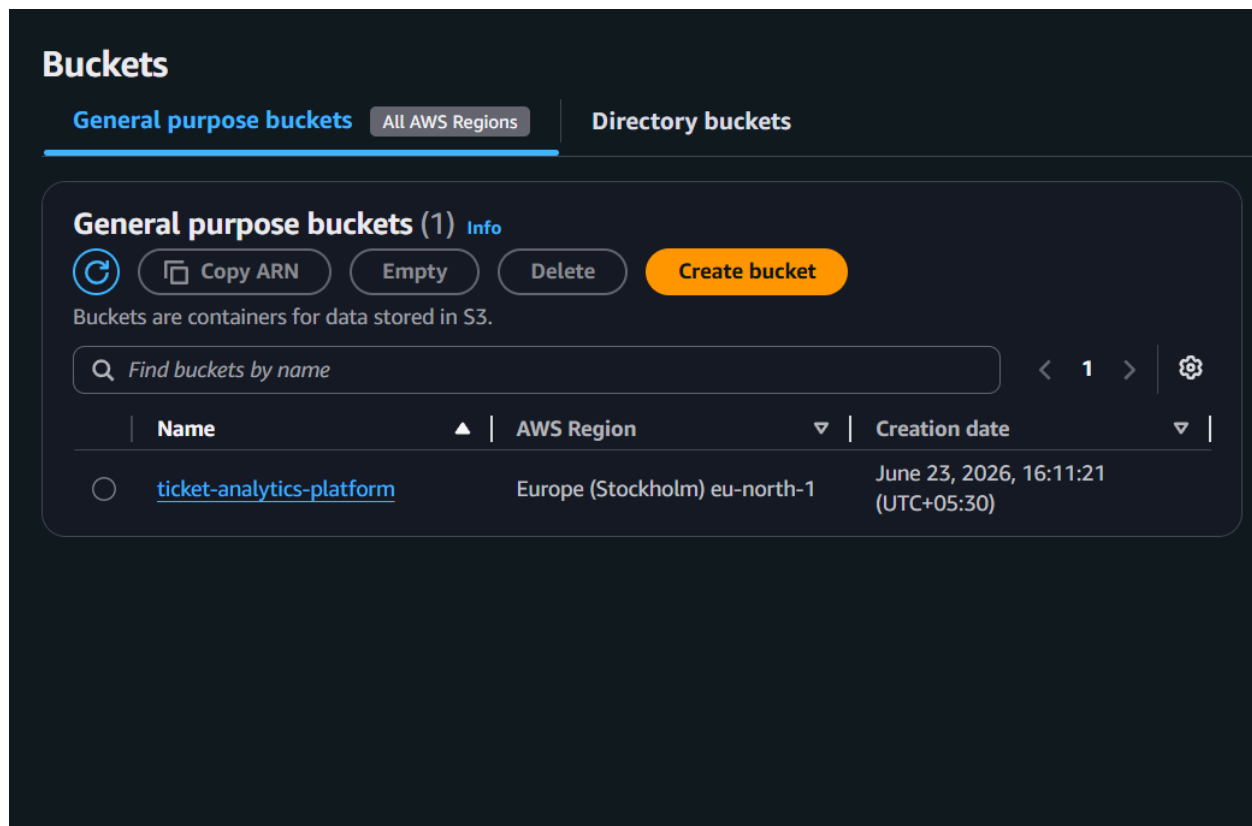
- SLA / resolution-target compliance
 - Workload distribution and efficiency by agent
 - Trend analysis by issue category over time
-

4) Cloud storage: AWS S3 Data Lake

Amazon S3 is used as the central Data Lake to store data throughout its lifecycle.

4.1 Bucket setup (high level)

- Create a dedicated S3 bucket for this project (naming convention based on your org/project standards).
- Enable appropriate IAM access (least privilege).
- (Optional) Enable versioning and server-side encryption (SSE-S3 or SSE-KMS).



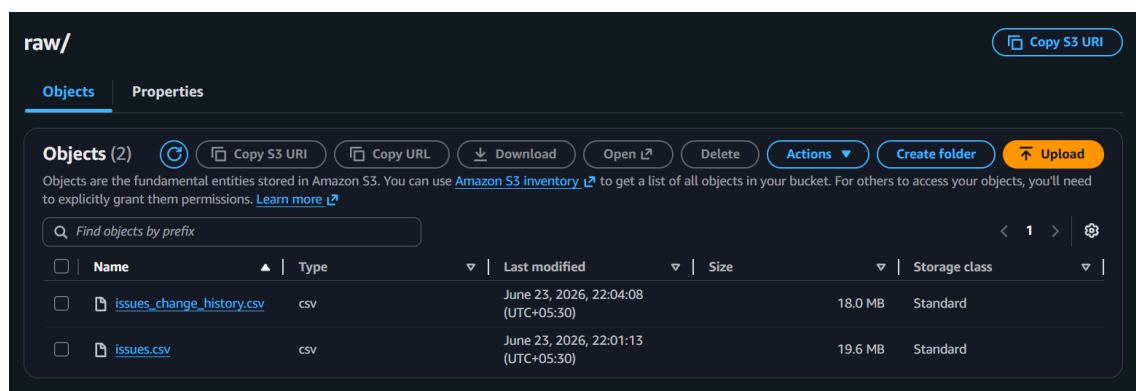
4.2 Folder layout (zone separation)

Recommended layout:

- **Bronze (Raw Zone)**

Stores unmodified raw CSVs from the source system.

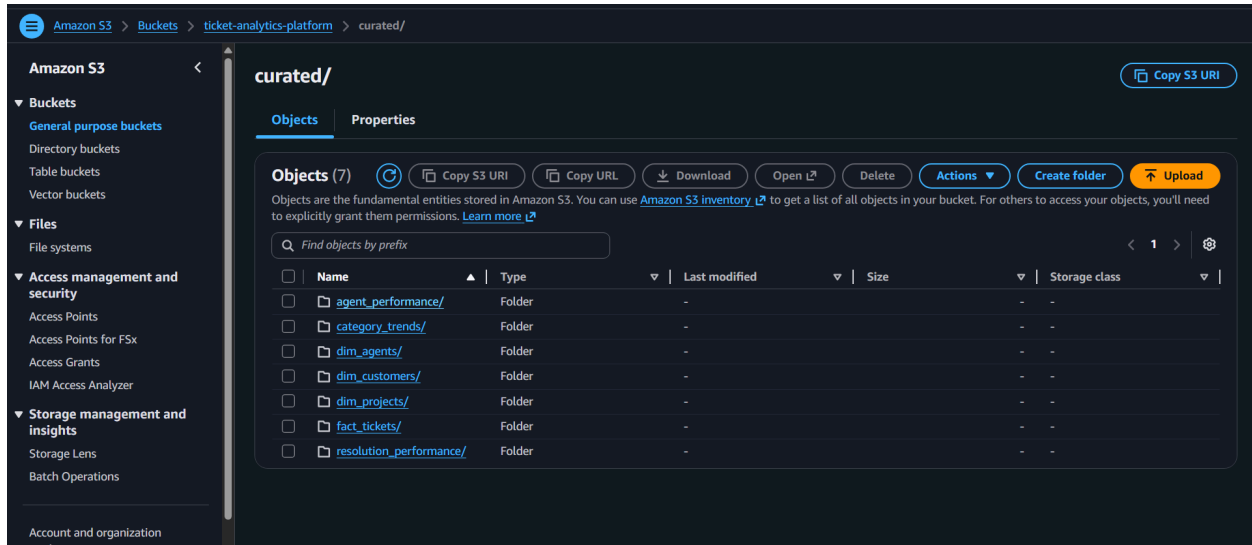
- Example: `s3://<bucket>/bronze/issues.csv`
- Example: `s3://<bucket>/bronze/issues_change_history.csv`



- **Gold (Curated Zone)**

Stores cleaned, modeled, analytics-ready data produced by PySpark, typically as **Parquet (Snappy)**.

- Example: `s3://<bucket>/gold/fact_tickets/`
- Example: `s3://<bucket>/gold/dim_agent/`



5) Architecture & ETL flow

5.1 End-to-end flow

1. Extract

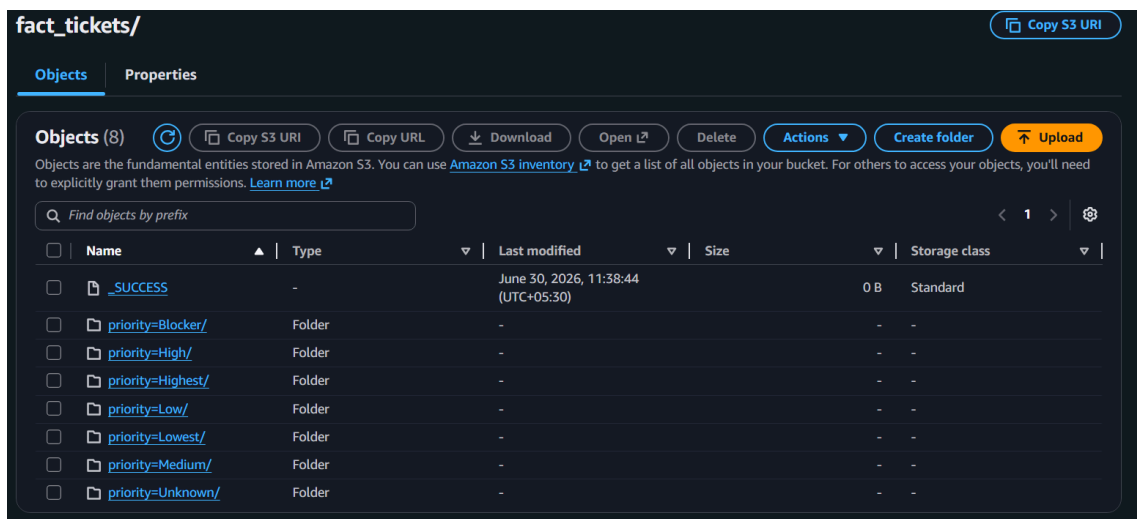
- Read raw CSV files from S3 (Bronze zone).

2. Transform (PySpark)

Core transformation responsibilities:

- Schema validation & type casting
- Data quality checks (null handling, anomaly filtering)
- Surrogate key generation for dimensions (as needed)
- Metric engineering, e.g.:
 - `resolution_duration_hours`

- `resolution_target_met` (priority-based threshold)
- Build a **star schema**:
 - `fact_tickets`
 - multiple dimension tables (e.g., agent, category, priority, date)



3. Load

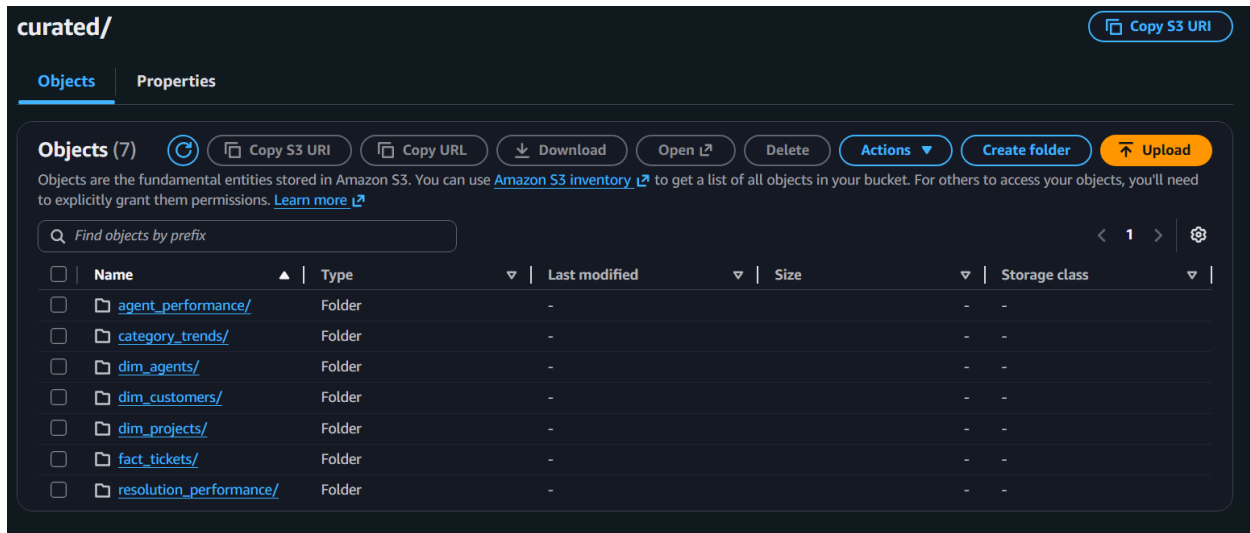
- Write curated outputs to S3 (Gold zone) as **Parquet (Snappy)** for storage and query efficiency.
- Load curated tables into **AWS RDS PostgreSQL** using JDBC for downstream SQL analytics and dashboard consumption.

5.2 Data model (star schema)

Typical modeled entities:

- **Fact**
 - `fact_tickets`: one row per ticket (or per ticket event depending on design)
- **Dimensions** (examples; adjust to your implementation)
 - `dim_agent`
 - `dim_category`
 - `dim_priority`

- `dim_date` (optional but common for time-series reporting)



5.3 Key engineered fields

- `resolution_duration_hours`

Computed time between created and resolved timestamps.

- `resolution_target_met`

Boolean derived by comparing resolution duration against thresholds by priority (e.g., P1 < X hours).

6) SQL analysis (RDS PostgreSQL)

After loading curated tables into PostgreSQL, run analytical queries to validate data and produce reporting datasets.

6.1 Validation & business metrics (placeholders)

Replace the following with your actual SQL tasks/queries and outputs:

- **Task 1:** SLA compliance by priority (met target vs total + compliance %)

```
postgres=> SELECT priority, COUNT(*) AS total_tickets, SUM(CASE WHEN resolution_target_met = TRUE THEN 1 ELSE 0 END) AS met_target, ROUND(
100.0 * SUM(CASE WHEN resolution_target_met = TRUE THEN 1 ELSE 0 END) / NULLIF(COUNT(*), 0), 2 ) AS compliance_rate_pct FROM f
act_tickets WHERE resolution_time IS NOT NULL GROUP BY priority ORDER BY priority;
priority | total_tickets | met_target | compliance_rate_pct
-----+-----+-----+-----
Blocker  |          641 |          0 |          0.00
High     |         4381 |        1211 |         27.64
Highest  |         2018 |          0 |          0.00
Low      |          501 |         226 |         45.11
Lowest   |           83 |          0 |          0.00
Medium   |        24249 |        6761 |         27.88
Unknown  |        33965 |          0 |          0.00
(7 rows)
```

- **Task 2:** Top agents by tickets resolved (most recent month window)

```
postgres=> WITH dataset_end AS ( SELECT MAX(resolution_time) AS max_ts FROM fact_tickets)SELECT da.agent_name, COUNT(ft.ticket_id) AS
tickets_resolved FROM fact_tickets ft JOIN dim_agents da ON ft.agent_id = da.agent_id, dataset_end WHERE ft.resolution_time >= DATE_TRUNC('mont
h', dataset_end.max_ts - INTERVAL '1 month') AND ft.resolution_time < DATE_TRUNC('month', dataset_end.max_ts)GROUP BY da.agent_name ORDER BY
tickets_resolved DESC LIMIT 5;
agent_name | tickets_resolved
-----+-----
4sgg      |          71
4hgkr     |          38
4lpo      |          30
4lsk      |          23
4hghq     |          22
(5 rows)

postgres=>
```

- **Task 3:** View — average resolution time by project

```
postgres=> CREATE OR REPLACE VIEW view_avg_resolution_by_project AS SELECT dp.project_name, COUNT(ft.ticket_id)
S total_tickets, ROUND(AVG(ft.resolution_duration_hours), 2) AS avg_resolution_hours FROM fact_tickets ft JOIN dim_projects dp ON ft.project
_id = dp.project_id WHERE ft.resolution_duration_hours IS NOT NULL GROUP BY dp.project_name ORDER BY avg_resolution_hours;SELECT * FROM view_av
g_resolution_by_project LIMIT 20;
CREATE VIEW
project_name | total_tickets | avg_resolution_hours
-----+-----+-----
1D83        |          1 |          0.06
C06HSD-0T   |          1 |          0.20
C11GR.N0    |          1 |          1.84
C02HK0_0    |          5 |         23.49
C12HX18AJ28AV.EW8X3D |         49 |         32.66
C11IOZC0U   |         19 |         35.85
C11GRZVZNV_ |         17 |         50.80
C11GR0D.NZ_ |         15 |         61.21
C11GR008_C  |          9 |         75.66
C08GH10V    |          4 |         75.79
C11HGND01   |          4 |         84.35
C11HGZVZNV_ |          8 |         96.30
C07GG_10U   |         17 |        101.26
C11HGZV_FF  |         10 |        102.66
C11GRZVZNV_ |        104 |        104.81
C11HG0229   |         26 |        106.39
C11HG0D.NZ  |         22 |        106.50
C11GRBX0    |          4 |        121.00
C11GRN0V    |          2 |        121.37
C11HGX0     |         16 |        131.00
-- More --
```

- **Task 4:** Agent speed ranking (avg resolution hours + rank)


```

postgres=> SELECT da.agent_name, ROUND(AVG(ft.resolution_duration_hours), 2) AS avg_resolution_hours, RANK() OVER (ORDER BY AVG(ft.resolution_duration_hours) ASC) AS rank_by_speed FROM fact_tickets ft JOIN dim_agents da ON ft.agent_id = da.agent_id WHERE ft.resolution_duration_hours IS NOT NULL GROUP BY da.agent_name ORDER BY rank_by_speed LIMIT 20;

```

agent_name	avg_resolution_hours	rank_by_speed
d1y10dygi	0.11	1
4hgks	0.25	2
av43.jd4aa.c3	0.33	3
w.nzwwz2ycz6dn28n	0.91	4
4qlr	2.88	5
4z8d4aa.c3	6.12	6
4hhgs	7.28	7
4hhop	11.74	8
4por	13.35	9
4lqr	18.01	10
-.3.jn2	18.30	11
u6yu9nygq	19.85	12
.wyx0.ygh	21.45	13
1n81u.adh	21.92	14
4lsg	24.19	15
4hhoo	26.62	16
t.yt2_0ygh	27.51	17
4hgo1	29.93	18
4hhgi	31.02	19
bzy0xaygh	32.22	20

(20 rows)
-- More --

- **Task 5:** Top reporters by ticket volume (by month)

```

postgres=> SELECT dc.reporter_name, DATE_TRUNC('month', ft.start_time) AS ticket_month, COUNT(*) AS ticket_count FROM fact_tickets ft JOIN dim_customers dc ON ft.customer_id = dc.customer_id GROUP BY dc.reporter_name, DATE_TRUNC('month', ft.start_time) HAVING COUNT(*) > 5 ORDER BY ticket_count DESC LIMIT 20;

```

reporter_name	ticket_month	ticket_count
z2w_x	2016-10-01 00:00:00	305
4qio	2017-04-01 00:00:00	118
4rqs	2020-11-01 00:00:00	86
4hgi	2016-03-01 00:00:00	84
4rqs	2020-12-01 00:00:00	81
t.y2xcuygs	2018-12-01 00:00:00	80
t.y2xcuyhp	2023-02-01 00:00:00	76
4qio	2015-12-01 00:00:00	75
4hgi	2016-02-01 00:00:00	73
4hlg	2016-12-01 00:00:00	71
z2w_x	2016-11-01 00:00:00	70
t.y2xcuyho	2023-03-01 00:00:00	70
4rqs	2020-09-01 00:00:00	67
t.y2xcuyhp	2023-01-01 00:00:00	66
4gor	2009-12-01 00:00:00	64
t.y2xcuyho	2022-12-01 00:00:00	63
4rqs	2020-10-01 00:00:00	63
t.y2xcuyhp	2022-12-01 00:00:00	63
t.y2xcuyhp	2022-10-01 00:00:00	61
4gis	2015-08-01 00:00:00	58

(20 rows)

- **Task 6:** Monthly resolved tickets + running total

```

postgres=> SELECT DATE_TRUNC('month', resolution_time) AS month, COUNT(*)
              AS monthly_resolved, SUM(COUNT(*)
    ) OVER ( ORDER BY DATE_TRUNC('month', resolution_time) )
              AS running_total FROM fact_tickets WHERE
    resolution_time IS NOT NULL GROUP BY DATE_TRUNC('month', resolution_time)ORDER BY month;

```

month	monthly_resolved	running_total
2013-01-01 00:00:00	1	1
2015-08-01 00:00:00	168	169
2015-09-01 00:00:00	263	432
2015-10-01 00:00:00	271	703
2015-11-01 00:00:00	347	1050
2015-12-01 00:00:00	326	1376
2016-01-01 00:00:00	419	1795
2016-02-01 00:00:00	399	2194
2016-03-01 00:00:00	2755	4949
2016-04-01 00:00:00	566	5515
2016-05-01 00:00:00	448	5963
2016-06-01 00:00:00	440	6403
2016-07-01 00:00:00	308	6711
2016-08-01 00:00:00	416	7127
2016-09-01 00:00:00	291	7418
2016-10-01 00:00:00	617	8035
2016-11-01 00:00:00	527	8562
2016-12-01 00:00:00	471	9033
2017-01-01 00:00:00	445	9478
2017-02-01 00:00:00	564	10042
2017-03-01 00:00:00	441	10483
2017-04-01 00:00:00	474	10957
2017-05-01 00:00:00	540	11497
2017-06-01 00:00:00	331	11828
2017-07-01 00:00:00	423	12251
2017-08-01 00:00:00	326	12577

-- More --

- **Task 7:** Category lookup (example: Project)

Result(RO) X

Search Results

Settings

Share

Fullscreen

Print

Refresh

Undo

Redo

Up

Export

Play

Eye

Cost: 420ms < 1 > Total 1

category

Project

category | Project

- **Task 8:** Resolution-time stats by priority (avg / min / max / ticket count)

fact_tickets					
Search Results					
	priority varchar(20)	avg_resolution_hours	total_tickets	min_hours	max_hours
>	Highest	485.93	2018	0.00	36717.60
>	High	591.17	4381	0.00	40255.47
>	Low	1107.47	501	0.00	29417.99
>	Blocker	1148.22	641	0.03	42164.17
>	Lowest	1304.83	83	0.02	24575.91
>	Medium	1515.50	24249	0.00	48926.57
>	Unknown	34598.41	33965	0.00	99220.54

- **Task 9:** View — ticket status distribution (count + %)

view_status_distribution			
Properties DATA ER Mock Manager postgres public			
SELECT * FROM view_status_distribution LIMIT 100			
Search Results			
	status varchar(60)	ticket_count bigint	percentage numeric
>	closed	56339	84.60
>	done	9712	14.58
>	waiting	219	0.33
>	in_progress	97	0.15
>	validation	86	0.13
>	resolved	61	0.09
>	pending_deployment	47	0.07
>	resolved_under_monitoring	18	0.03
>	under_review	5	0.01
>	to_do	3	0.00
>	in_review	2	0.00
>	approved	2	0.00
>	reopened	1	0.00
>	cancelled	1	0.00

- **Task 10:** Detailed results (project + agent + avg resolution hours)

Result(RO) X		
+ - Q Search Results ⚙️ 📧 👤 + + 🗑️ ↺ ↻ ⬆️ ⬆️ Export ▶️ 👁️ Cost: 405ms < 1 2 3 4 ... 18 > Total 344		
Q	project_name	agent_name avg_resolution_hours
>	ADD2	4sig 1064.71
>	C00D3DZWJ28AV.EW8X3	4hggg 0.22
>	C01GG0XA	4hgqg 143.84
>	C01GG1_3_	4qkp 130.69
>	C01GGD_0	4z8d4aa.c3 6.12
>	C01GGX0Z2	4gsp 282.68
>	C01GGZ-3	4kgq 0.11
>	C01HG826	4kqo 0.36
>	C01HQD_0	4prg 1.74
>	C01HQND01	4hhgk 118.87
>	C02GG_0	4ikk 595.21
>	C02GG0_0	4lhl 2.33
>	C02GG00U	4hgok 21.75
>	C02GG020	4kgr 13.41
>	C02GG0_D0	4ohk 86.70
>	C02GG0W_	0ny0x93ygi 118.65
>	C02GG0X93	4prg 66.64
>	C02GG 1 1	4ikk 47.14

7) Business use cases (Streamlit visualizations)

The Streamlit dashboard connects to the RDS PostgreSQL database using **SQLAlchemy** and presents interactive reporting across three core use cases.

Use case 1: Resolution performance analysis

Business goal

Understand what share of tickets are resolved within target timelines, broken down by priority and over time.

ETL / analytics flow

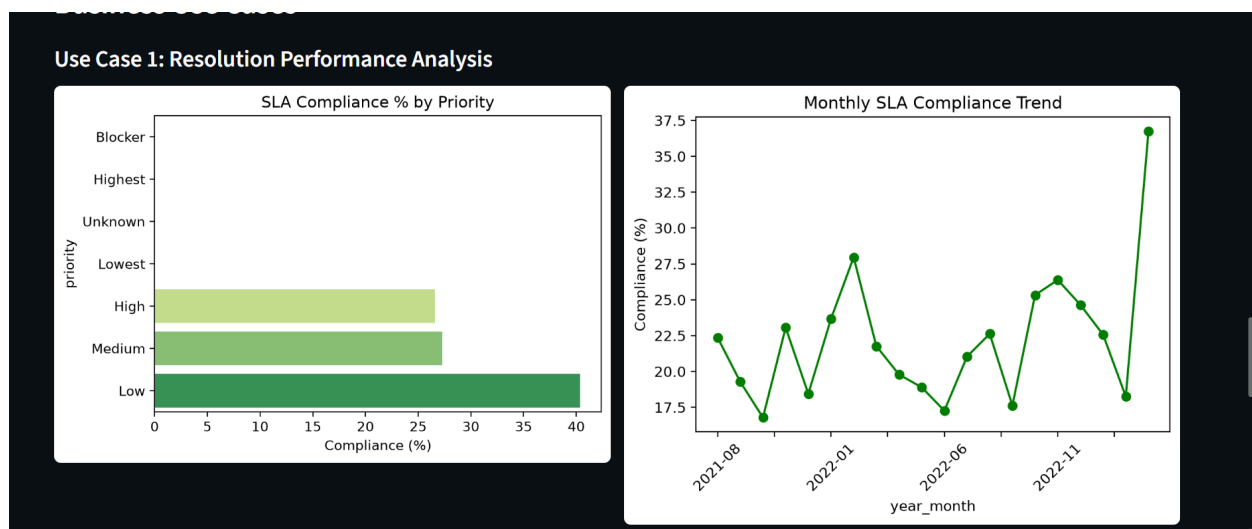
1. Compute `resolution_target_met` per ticket using priority-specific thresholds.
2. Aggregate compliance rate by **priority** and **month**.
3. Write aggregated results to RDS (example output:
`analytics.resolution_performance`).

Dashboard outputs

- Bar chart: compliance rate by priority
- Line chart: monthly compliance trend
- KPI: overall compliance percentage

Expected insight

Identifies which priority levels frequently miss targets, enabling targeted staffing and process improvements.



Use case 2: Agent workload & performance

Business goal

Understand distribution of ticket load and resolution efficiency across agents.

ETL / analytics flow

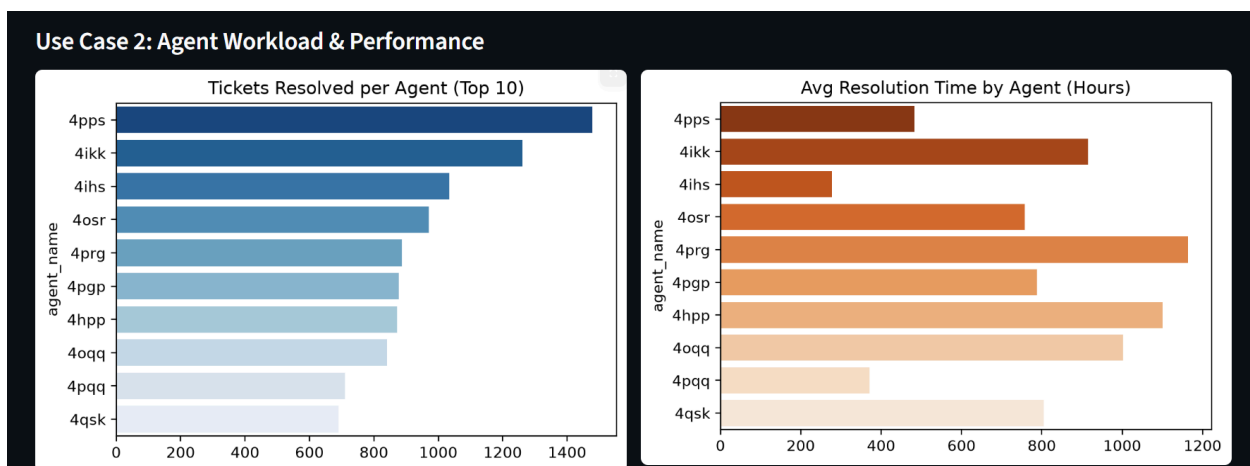
1. Aggregate `fact_tickets` by `agent_id`:
 - ticket count
 - average `resolution_duration_hours`
2. Rank agents by tickets resolved (window function).
3. Write ranked metrics to RDS (example output: `analytics.agent_performance`).

Dashboard outputs

- Bar chart: tickets resolved per agent
- Trend chart: average resolution time per agent (over time)
- Distribution chart: workload spread (identify overloaded vs underutilized)

Expected insight

Highlights top performers and workload imbalance to guide coaching, staffing, and process improvements.



Use case 3: Ticket category trends

Business goal

Identify recurring or fast-growing issue categories that may indicate product gaps or documentation needs.

ETL / analytics flow

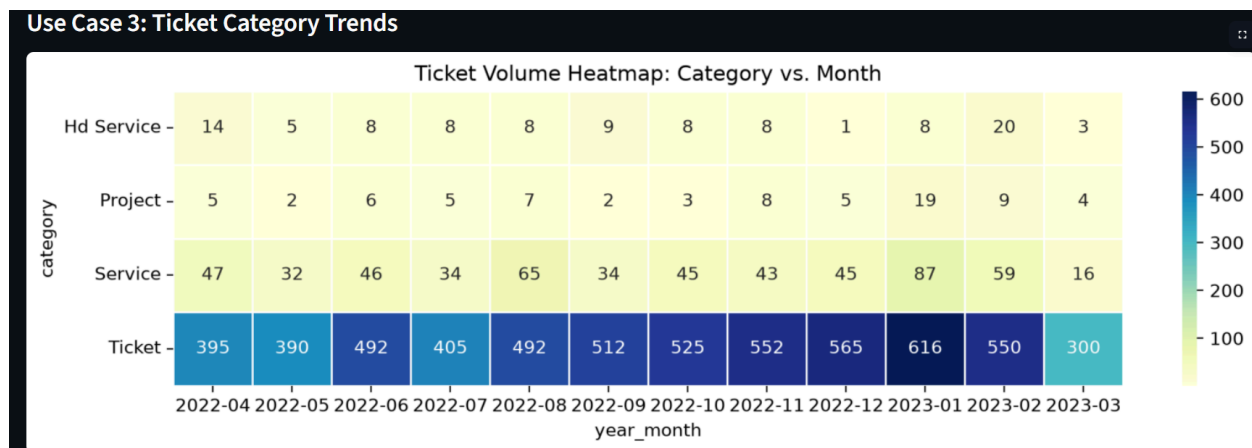
1. Aggregate tickets by **category** and **month**:
 - ticket volume
 - average `resolution_duration_hours`
2. Compute month-over-month (MoM) % change in volume per category.
3. Write category trends to RDS (example output: `analytics.category_trends`).

Dashboard outputs

- Heatmap: category vs month ticket volume
- Line chart: fastest-growing categories
- KPI: top recurring category

Expected insight

Surfaces categories that warrant product fixes or improved self-service documentation.



8) Conclusion

This project delivers a working, automated analytics pipeline for AWS support ticket data, taking raw, unstructured helpdesk exports and turning them into a governed, queryable, and visualized data product. By combining PySpark for scalable transformation, a Lambda-triggered event-driven architecture for automation, S3 for durable zone-separated storage, and RDS PostgreSQL as the analytics layer, the pipeline removes the need for manual pipeline kick-off or spreadsheet-based reporting.

The resulting dashboards give support leadership direct visibility into three previously invisible dimensions of their operation: whether tickets are meeting resolution-time targets by priority, how workload and efficiency are distributed across agents, and which issue categories are trending upward in volume — a signal for potential product or documentation gaps. Each of these outputs is backed by validated, deduplicated, and reconciled data, with 15 logged test cases

covering edge cases like duplicate ticket IDs, out-of-order timestamps, unassigned tickets, and idempotent re-runs.

Beyond the technical deliverable, the project reinforced practical, production-relevant skills: reconstructing current-state records from an event/change-history log, designing a star schema for time-series and workload reporting, building idempotent load logic keyed on a natural identifier, and wiring an event-driven trigger (S3 → Lambda → PySpark) rather than relying on manual orchestration.

The architecture is intentionally scoped to a small, defensible set of AWS services, demonstrating that a production-style analytics platform can be built without unnecessary infrastructure complexity